

ASSIGNMENT



Inspiring Excellence

Submitted to: Souvik Ghosh (SVKG) Sir

Submitted by: Sheikh Md. Tahmid Hossain

ID: 22299522

Section: 26

Assignment No: 3

Date: 26 / 12 / 25

Course Code: CSE340

Answer to the Question No. 1

Given,

0x ABB 9609

1010 1011 1011 1001 0110 0000 1001

Here,

MSB sign = 1 (negative)

Exponent = 010101

Now,

$$\text{Bias} = 2^6 - 1 = 31$$

$$\therefore \text{Actual} = 21 - 31 = -10$$

$$\therefore \text{Value} : 1 + (1 + 2^{-1} + 2^{-2} + 2^{-4} + 2^{-5} + 2^{-6} + 2^{-7} + 2^{-11} + 2^{-12} + 2^{-13} + 2^{-21}) \times 2^{10}$$

$$= 1.81842 \times 10^{-3}$$

$$= -0.001812$$

Answer to the Question No. 2

(a)

$$\text{Given, } (50.7864)_{10} = (110010.11001)_2$$

$$= (1.1001011001) \times 2^5$$

$$(79.83)_{10} = (1001111.11010)_2$$

$$= (1.0011111010) \times 2^6$$

$$(29.58)_{10} = (11101.10010)_2$$

$$= (1.11011001) \times 2^4$$

$$(50.7864 + 79.83 - 29.58)_{10} = (101.0369)_{10}$$

$$\text{Now, } (101.0369)_{10} = (1100101.00001001011)_2$$

$$= (1.10010100001001011) \times 2^6$$

(b)

Given,

$$(64.2486)_{10} = (1000000.00111)_2$$

$$= (1.00000000111) \times 2^6$$

$$(49.1832)_{10} = (110001.00101)_2$$

$$= 01.10000100101 \times 2^6$$

$$\therefore 64.2486 \times 49.1832 = 1.100010101111111 \dots$$

$$00111101 \times 2^{11}$$

Answer to the Question No. 3

Here,

$$28.4810 - (-4.0210) = 28.4810 + 4.0210$$

Now,

$$(28.4810)_{10} = (11100.01111)_2 = 1.110001111 \times 2^4$$

$$(-4.0210)_{10} = (100.00000)_2 = 1.00000 \times 2^2$$

$$= 0.0100000 \times 2^4$$

$$\therefore (28.4810)_{10} + (4.0210)_{10} = 10.00001111 \times 2^4$$

$$= 1.000001111 \times 2^5$$

Here, final exponent = 5

$$\therefore 1 \leq 5 \leq 127$$

$\therefore$ No overflow or underflow

Answer to the Question No. 4

(a)

Bias is added for the following reasons!

- Allows floating point numbers to be compared using standard integer comparison hardware
- Avoids the need for a separate sign bit for the exponent.

(b)

Optimized multiplication improves efficiency

by:-

- (i) Reduces partial products
- (ii) Reduces clock cycle delay to calculate products.

Answer to the Question No. 5

Instruction	i) Function	(ii) Syntax	(iii) Example
(i) fadd.s	Adds two single-precision (32 bit) floating point numbers	fadd.s rd, rs1, rs2	fadd.s f0, f1, f2
(ii) fsub.s	Subtracts two single-precision (32 bit) floating-point numbers	fsub.s rd, rs1, rs2	fsub.s f3, f4, f5
(iii) fmul.s	Multiplies two single-precision (32-bit) floating point numbers	fmul.s rd, rs1, rs2	fmul.s f6, f7, f8
(iv) fdiv.s	Divides two single-precision (32 bit) floating numbers	fdiv.s rd, rs1, rs2	fdiv.s f9, f10, f11
(v) fsqrt.s	Calculates the square root of a single-precision (32-bit) floating-point number	fsqrt.s rd, rs1	fsqrt.s f12, f13
(vi) feq.d	Compares two double-precision numbers for equality; sets integer rd to 1 if equal, 0 if not	feq.d rd, rs1, rs2	feq.d x5, f14, f15
(vii) fle.d	Compares two double-precision numbers; sets integer rd to 1 if $rs1 \leq rs2$	fle.d rd, rs1, rs2	fle.d x6, f16, f17

Instruct	i) Function	ii) Syntax	iii) Example
vii) flt.d	Compares two single-precision numbers; sets integer rd to 1 if $rs1 < rs2$	flt.s rd, rs1, rs2	flt.s x7, f18, f19
ix) mul	Multiplies two 32-bit integers; stores the lower 32 bits of the product	mul rd, rs1, rs2	mul x5, x10, x11
x) mulh	Multiplies two signed 32-bit integers; stores the higher 32 bits	mulh rd, rs1, rs2	mulh x6, x10, x11
xi) mulhu	Multiplies two unsigned 32-bit integers; stores the higher 32 bits	mulhu rd, rs1, rs2	mulhu x7, x12, x13
xii) mulhsu	Multiplies one signed (rs1) and one unsigned (rs2) integer; stores the higher 32 bits	mulhsu rd, rs1, rs2	mulhsu x8, x10, x12
xiii) flw	Loads a single-precision (32-bit) float from memory into a float register	flw rd, offset(rs1)	flw f20, 0(x10)
xiv) fsd	stores a double precision (64-bit) float from a float register into memory	fsd rs2, offset(rs1)	fsd f22, 8(x10)

Answer to the Question No. 6

'mulh' is useful for overflow checking because it gives us the high half of the full product.

Doing $low = mul(a, b)$ and $high = mulh(a, b)$ (signed)
/ $mulhu(a, b)$ (unsigned)

Checking unsigned overflow

if $high \neq 0 \rightarrow$ product doesn't fit in
64 bits (overflow)

Example: $(2^{64} - 1) * 2 \rightarrow low = 0x \dots,$
 $high = 1 \rightarrow$ Overflow

Checking signed overflow,

Computing $signext = low \gg 63$; if $high \neq signext$
overflow.

Example: $(2^{63} - 1) * 2 \rightarrow low = 0x \dots$ (negative)
 $high = 0 \neq signext = -1$
(overflow)

Answer to the Question No. 7

feq.d x5, f1, f2

bneq x5, jump Equal

j jump Not Equal

jump Equal:

jump Not Equal:
